

BCC2001

— ORIGIN
'warn'.

monitor •

thing. 2 a software
duties. 3 a software
picture for use in
process which

Diego Bertolini

diegobertolini@gmail.com

Introdução à Ling. C.

- **Aula Anterior:**
 - Introdução à linguagem C.
- **Aula de Hoje:**
 - Operadores aritméticos, lógicos e relacionais.

Operador de Atribuição “=”

- A Atribuição “ = ” é uma das operações mais usadas na programação ;
- Responsável por atribuir determinado valor a uma variável ;
- Em linguagem C, o uso do operador “=” segue esta forma geral:

nome_da_variavel = expressao ;

- Por expressão, entende-se: Valores, Variáveis, Constantes ou Chamadas de Funções ;

Exemplos:

```
1  #include<stdio.h>
2  #include<math.h>
3
4  // Já visto em aula
5  #define z 5.0
6  // Constante assim como define
7  // const tipo_da_constante nome_da_constante = valor_da_constante
8  const float zz = 9 ;
9
10 int main(){
11     float x ;
12     float y = 3 ;
13
14     x = 5 ;
15     x = z ;
16     x = y + 5 ;
17     x = sqrt(zz) ;
18
19     printf("Valor de X: %f \n", x);
20     printf("Valor de Y: %f \n", y);
21     printf("Valor de Z: %f \n", z);
22
23     return (0);
24
25 }
```

usadas na

uma

e esta

S,

Exemplos:

```
1  #include<stdio.h>
2  #include<math.h>
3
4  // Já visto em aula
5  #define z 5.0
6  // Constante assim como define
7  // const tipo_da_constante nome_da_constante = valor_da_constante
8  const float zz = 9 ;
9
10 int main(){
11     float x ;
12     float y = 3 ;
13     // ERRADO!
14     x + 5 = 2 ;
15     x + y = 5;
16     5 = x ;
17
18     printf("Valor de X: %f \n", x);
19     printf("Valor de Y: %f \n", y);
20     printf("Valor de Z: %f \n", z);
21
22     return (0);
23
24 }
```

Operador de Atribuição “=”

- A Linguagem C suporta Múltiplas Atribuições ;

```
1  #include<stdio.h>
2  #include<math.h>
3
4
5  int main(){
6      float x, y, z;
7      int a , b , c = 33 ;
8
9      x = y = z = 5 ;
10     a = b = c ;
11
12     printf("Valor de X: %f \n", x);
13     printf("Valor de Y: %f \n", y);
14     printf("Valor de Z: %f \n", z);
15
16     printf("Valor de X: %d \n", a);
17     printf("Valor de Y: %d \n", b);
18     printf("Valor de Z: %d \n", c);
19
20     return (0);
21
22 }
```


Operador de Atribuição “=”

- A Linguagem C também permite atribuição entre dois tipos básicos diferentes ;
- O compilador converte automaticamente o valor do lado direito para o tipo do lado esquerdo do comando de atribuição “=”;
- Durante a etapa de conversão de tipos, pode haver perda de informação.

Operador de Atribuição “=”

- Na conversão de tipos, durante a atribuição, pode haver perdas!

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 65 ;
5      float f = 25.1 ;
6      char ch ;
7      // converte para Tabela ASCII
8      // 65 == A
9      ch = x ;
10     printf("Valor de CH: %c \n", ch);
11
12     // Recebe a parte inteira de f ;
13     x = f ;
14     printf("Valor de X: %d \n", x);
15
16     // f recebe o valor 8 bits convertido para float
17     f = ch ;
18     printf("Valor de F: %f \n", f);
19     // f recebe o valor de x
20     f = x ;
21     printf("Valor de F: %f \n", f);
22
23     return (0);
24
25 }
```

```
Valor de CH: A
Valor de X: 25
Valor de F: 65.000000
Valor de F: 25.000000
```

```
Process returned 0 (0x0)
```


Operadores Aritméticos

+	(soma) ;
-	(subtração) ;
*	(multiplicação) ;
/	(divisão) ;
%	(resto da divisão inteira) .

Operadores Aritméticos

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 10, y = 20, z, k, e ;
5
6      z = x * y ;
7      printf("z = %d \n", z) ;
8
9      z = y / 10 ;
10     printf("z = %d \n", z) ;
11     printf("x + y = %d \n", x+y) ;
12
13     printf("k = %d\n", 10 % 2) ;
14     k = 10 % 3 ;
15     printf("k = %d\n", k );
16
17     printf("E = %d \n", (x*x) + (y / 2) + (k * 9)) ;
18
19     return (0);
20 }
```

Operador de Atribuição “=”

- Podemos devolver o resultado de uma expressão a uma variável ;
- Ou para um outro comando ou função que espere receber um valor do mesmo tipo do resultado da operação.
- De modo geral, os operadores aritméticos são operadores binários (atuam sobre dois valores);
- Os operadores de Adição e Subtração podem ser unários ;

$x = - y ;$

$X = (-1) * y ;$

Operador de Atribuição “=”

Em uma operação utilizando o operador de quociente (/), se o numerador e o denominador forem números inteiros, por padrão o compilador retornará apenas a parte inteira da divisão ;

```
1  #include<stdio.h>
2
3  int main(){
4      float x ;
5
6      x = 5 / 4 ;
7      printf("x = %f \n", x);
8
9      x = 5 / 4.0 ;
10     printf("x = %f \n", x);
11
12     return (0);
13 }
14
```

```
x = 1.000000
x = 1.250000
```

Operadores Relacionais

- Os operadores relacionais são aqueles que operam sobre dois valores (valores, variáveis, constantes ou chamadas de funções) e/ou expressões e verificam a magnitude (qual é maior ou menor) e/ou igualdade entre eles.
- Os operadores relacionais são operadores de comparação de valores ;
- A linguagem C possui 6 operadores relacionais:

Operadores Relacionais

```
==      (igual a);  
!=      (diferente de);  
>       (maior do que);  
<       (menor do que);  
>=     (maior ou igual a);  
<=     (menor ou igual a).
```


Operadores Relacionais

- Como resultado, esse tipo de operador retorna:
 - O valor UM (1), se a expressão relacional for considerada VERDADEIRA ;
 - O valor ZERO (0), se a expressão relacional for considerada FALSA ;

Operadores Relacionais

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 5, y = 3;
5
6      printf("Resultado de x > 4: %d \n", x > 4);
7      printf("Resultado de x == 4: %d \n", x == 4);
8      printf("Resultado de x != y: %d \n", x != y);
9      printf("Resultado de x != (y+2): %d \n", x != y);
10
11     return (0);
12 }
13
```

```
Resultado de x > 4: 1
Resultado de x == 4: 0
Resultado de x != y: 1
Resultado de x != (y+2): 1
```

Operadores Lógicos

- Certas situações não podem ser modeladas apenas utilizando operadores aritméticos e relacionais ;
- Um exemplo bem simples disso é saber se determinada variável esta dentro de uma faixa de valores ;
- Por exemplo:

$$0 < x < 10$$

Para modelar esse tipo de situação, a linguagem C possui um conjunto de três operadores Lógicos.

Operadores Lógicos

!	(negação - not - não) ;
	(disjunção - or - ou) ;
&&	(conjunção - and - e) .

Operadores Lógicos

- **Operador E (&&):** A expressão só é verdade se ambas as expressões unidas por esse operador também forem.
- **Operador OU (||):** A expressão resultante é verdadeira se alguma das expressões unidas por esse operador também for.
- **Operador NEGAÇÃO (!):** Inverte o valor lógico da expressão à qual se aplica.
- Os operadores Lógicos atuam sobre valores lógicos e retornam um valor lógico
 - **1 : Se a expressão for verdadeira ;**
 - **0 : Se a expressão for falsa;**

Operadores Lógicos

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 5, y = 3, r;
5
6      r = (x > 2) && (y < x) ;
7      printf("Resultado: %d \n", r);
8
9      r = (x % 2 == 0) && (y < 0) ;
10     printf("Resultado: %d \n", r);
11
12     r = (x > 2) || (y > x) ;
13     printf("Resultado: %d \n", r);
14
15     r = (x % 2 == 0) || (y < 0) ;
16     printf("Resultado: %d \n", r);
17
18     r = ! (x > 2) ;
19     printf("Resultado: %d \n", r);
20
21     r = ! (x > 7) && (x > y);
22     printf("Resultado: %d \n", r);
23
24
25     return (0);
26 }
```

```
Resultado: 1
Resultado: 0
Resultado: 1
Resultado: 0
Resultado: 0
Resultado: 1
```

Tabela Verdade

A	B	A E B	A OU B	NÃO A	NÃO B
<i>VERDADEIRO</i>	<i>VERDADEIRO</i>	<i>VERDADEIRO</i>	<i>VERDADEIRO</i>	<i>FALSO</i>	<i>FALSO</i>
<i>VERDADEIRO</i>	<i>FALSO</i>	<i>FALSO</i>	<i>VERDADEIRO</i>	<i>FALSO</i>	<i>VERDADEIRO</i>
<i>FALSO</i>	<i>VERDADEIRO</i>	<i>FALSO</i>	<i>VERDADEIRO</i>	<i>VERDADEIRO</i>	<i>FALSO</i>
<i>FALSO</i>	<i>FALSO</i>	<i>FALSO</i>	<i>FALSO</i>	<i>VERDADEIRO</i>	<i>VERDADEIRO</i>

Operadores de Atribuição Simplificada

- Muitos operadores são sempre usado em conjunto com o operador de atribuição ;
- Para tornar esta tarefa mais simples, a Linguagem C permite simplificar algumas expressões

Operadores de Atribuição Simplificados.

Op.	Significado	Exemplo
<code>*=</code>	Aritmético de atribuição (multiplicação)	<code>N*=2;</code> equivale a <code>N=N*2;</code>
<code>/=</code>	Aritmético de atribuição (divisão)	<code>N/=4;</code> equivale a <code>N=N/4;</code>
<code>%=</code>	Aritmético de atribuição (módulo)	<code>N%=5;</code> equivale a N igual ao resto da divisão de N por 5;
<code>+=</code>	Aritmético de atribuição (adição)	<code>N+=3;</code> equivale a <code>N=N+3;</code>
<code>-=</code>	Aritmético de atribuição (subtração)	<code>N-=2;</code> equivale a <code>N=N-2;</code>

Operadores de Pré e Pós Incremento

- Além dos operadores simplificados, a linguagem C também possui operadores de incremento (++) e decremento (--). Esses operadores podem ser utilizados sempre que for necessário somar uma unidade (incremento) ou subtrair uma unidade (decremento) a determinado valor.
 - Incremento (++)
 - Decremento (–)

Exemplos:

- $x++ \rightarrow x = x + 1$
- $y-- \rightarrow y = y - 1$
- **$++x \rightarrow x = x + 1$**
- **$--y \rightarrow y = y - 1$**

Operadores de Pré e Pós Incremento

- Note que este operador pode ser usado antes ou depois do nome da variável, com uma diferença significativa:
 - ++x (pré-incremento): **Soma** +1 a variável x **antes** de utilizar seu valor ;
 - x++ (pós-incremento): **Soma** +1 a variável x **depois** de utilizar seu valor ;
 - --x (pré-decremento): **Subtrai** -1 a variável x **antes** de utilizar seu valor ;
 - x-- (pós-decremento): **Subtrai** -1 a variável x **depois** de utilizar seu valor ;

Operadores de Pré e Pós Incremento

$y = ++x$;

é equivalente à:

- $x = x + 1$
- $y = x$

$y = x++$

é equivalente à :

- $y = x$;
- $x = x + 1$;

Operadores de Pré e Pós Incremento

- Essa diferença de sintaxe no uso do operador não tem importância se o operador for usado sozinho;

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 10 , y = 10 , a = 1, b = 1 ;
5
6      x++ ;
7      ++y ;
8
9      a-- ;
10     --b ;
11
12     printf("Pós-Incremento: %d \n", x);
13     printf("Pré-Incremento: %d \n", y);
14     printf("Pós-Decremento: %d \n", a);
15     printf("Pré-Decremento: %d \n", b);
16
17     return (0);
18 }
19
```

```
Pós-Incremento: 11
Pré-Incremento: 11
Pós-Decremento: 0
Pré-Decremento: 0
```

Operadores de Pré e Pós Incremento

- Porém, se esse operador for utilizado dentro de uma expressão aritmética, a diferença entre os dois operadores será evidente;

```
1  #include<stdio.h>
2
3  int main(){
4      int x = 10 , y = 10, pre, pos ;
5
6      pos = x++ ;
7      pre = ++y ;
8      //Atribui depois incrementa
9      printf("Pós-Incremento: %d \n", pos);
10     //Incrementa depois atribui
11     printf("Pré-Incremento: %d \n", pre);
12
13     return (0);
14 }
15
```

```
Pós-Incremento: 10
Pré-Incremento: 11
```


Modeladores de Tipo (*casts*)

Modeladores de tipo ou (*type cast*) são uma forma explícita de conversão de tipo, na qual o tipo a ser convertido é explicitamente definido dentro de um programa. Em linguagem C, o uso do modelador de tipo, segue a forma geral:

(nome_do_tipo) expressão

- Um modelador de tipo é definido pelo próprio nome_do_tipo entre parênteses. Ele é colocado à frente de uma expressão e tem como objetivo forçar o resultado da expressão a ser de um tipo especificado

Modeladores de Tipo (*casts*)

```
1  #include<stdio.h>
2
3  int main(){
4      float x, y, f = 65.5 ;
5
6      x = f / 10.0 ;
7      y = (int) (f/10.0);
8
9      printf("x = %f \n", x);
10     printf("y = %f \n", y);
11
12     return (0);
13 }
14
```

```
x = 6.550000
y = 6.000000
```

Modeladores de Tipo (*casts*)

```
1  #include<stdio.h>
2
3  int main(){
4      float rFloat, rFloatc ;
5      int a = 10, b = 3 , rInt, rIntc ;
6
7      rInt = a / b ;
8      rFloat = a / b ;
9
10     rIntc = (float) a / b ;
11     rFloatc = (float) a / b ;
12
13     printf("rInt = %d \n", rInt);
14     printf("rFloat = %f \n", rFloat);
15
16     printf("rIntc = %d \n", rIntc);
17     printf("rFloatc = %f \n", rFloatc);
18
19     return (0);
20 }
21
```

```
rInt = 3
rFloat = 3.000000
rIntc = 3
rFloatc = 3.333333
```

Operador Vírgula (,)

Na Linguagem C, o operador vírgula pode ser utilizado de duas maneiras ;

- Como pontuação para separar elementos de uma função:

- `int minha_funcao (int a , int b, float r)`

- Para determinar uma lista de expressões que devem ser executadas sequencialmente:

- `X = (y = 2 , y + 3) ;`

Nesse caso, as expressões serão executadas da esquerda para direita;

Precedência de Operadores

A Linguagem C contém muitos operadores. Consequentemente o uso de muitos operadores em uma expressão pode tornar confuso a sua interpretação. Por esse motivo a linguagem C possui uma série de regras de precedência de operadores.

Isso permite que o compilador possa decidir corretamente qual a ordem que os operadores serão executados ;

Seguem basicamente as regras matemáticas ;

Pode-se utilizar parênteses para forçar o compilador a executar uma parte da expressão antes das demais ;

Tabela dos mais comuns.

Categoria	Operador	Exemplo
Aritmético (unário)	++ --	a++
Aritmético (unário)	+ - ++ --	++a, -a
Aritmético	* / %	a / 2
Aritmético	+ -	a + 5
Relacional	< <= > >=	a > 5
Relacional	== !=	a == 5
Lógico	&&	a && b
Lógico		a c
Atribuição	= += -= *= /= %=	a += b

Perguntas??

